

Documentation technique



Image Pixabay

Réflexion initiale sur les technologies utilisées :

L'application EcoRide repose sur une stack technique volontairement légère et maîtrisable

Technologies choisies :

Front End :

- **HTML5 / CSS3 / Bootstrap 5** : structure des pages et mise en forme responsive.
- **JavaScript** : interactivité (fetch), chargement dynamique de page, système de routage dynamique, gestion des événements, création de formulaires dynamiques.
- **Chart.js** : pour l'affichage des statistiques graphiques.

Back End :

- **PHP vanilla** : J'ai choisi PHP vanilla plutôt qu'un framework pour mieux comprendre les bases du langage (et donc mieux visualiser le fonctionnement d'un framework), avoir un contrôle total sur la logique du projet et éviter la charge supplémentaire d'un framework.
- **PDO** : exécution sécurisée des requêtes SQL via des requêtes préparées.

Bases de données :

- **MySQL (relationnelle)** : pour la gestion des données principales : utilisateurs, covoiturages, véhicules, réservation, préférences.
- **MongoDB (NoSQL)** : pour logs de la plateforme (avis, crédits, suspensions...) et me baser sur ces logs pour les statistiques.

Déploiement :

- **Heroku** : hébergement du site PHP avec base de données MySQL (via **JawsDB**) et MongoDB (via **Atlas**).
- **Composer** : gestion des dépendances PHP (phpmailer, mongodb, dotenv).

Configuration de l'environnement de travail :

Local :

- **XAMPP** : Apache + MySQL pour le serveur local et phpMyAdmin pour la gestion de la base de données.
- **MongoDB Compass** : avec l'ensemble des collections logs NoSQL.
- **Visual Studio Code (VSCode) comme IDE avec les extensions** :
 1. GitGraph pour le suivi des commits,
 2. Live Sass Compiler pour les compilations SCSS en CSS (utilisé pour personnaliser Bootstrap),
 3. PHP Intelephense et PHP Namespace Resolver pour programmer et importer les classes plus facilement,
 4. MongoDB pour VSCode me permettant d'accéder aux collections depuis l'IDE,
 5. PHP Server pour ouvrir un serveur local et PHPUnit pour les tests.

Dépendances installées via Composer :

- **phpmailer/phpmailer (v6.9)** : pour l'envoi de mails,
- **mongodb/mongodb (v1.20)** : client MongoDB natif PHP,
- **vlucas/phpdotenv (v5.6)** : gestion sécurisée des variables d'environnement.

Environnement distant :

- **Git via GitHub** : pour versionner mon projet, travailler sur différentes branches (main, dev, feature/*) et revenir sur des versions précédentes quand nécessaire.
- **Heroku** : déploiement de l'application (connecté à GitHub).
- **JawsDB MySQL** : add-on de Heroku pour la base de données relationnelle.

- **MongoDB Atlas** : base de données NoSQL distante pour les logs.
- **MySQL Workbench** : interface pour gérer JawsDB MySQL.

Schemas MCD de la base de données relationnelle

(réalisés sur dbdiagram.io)

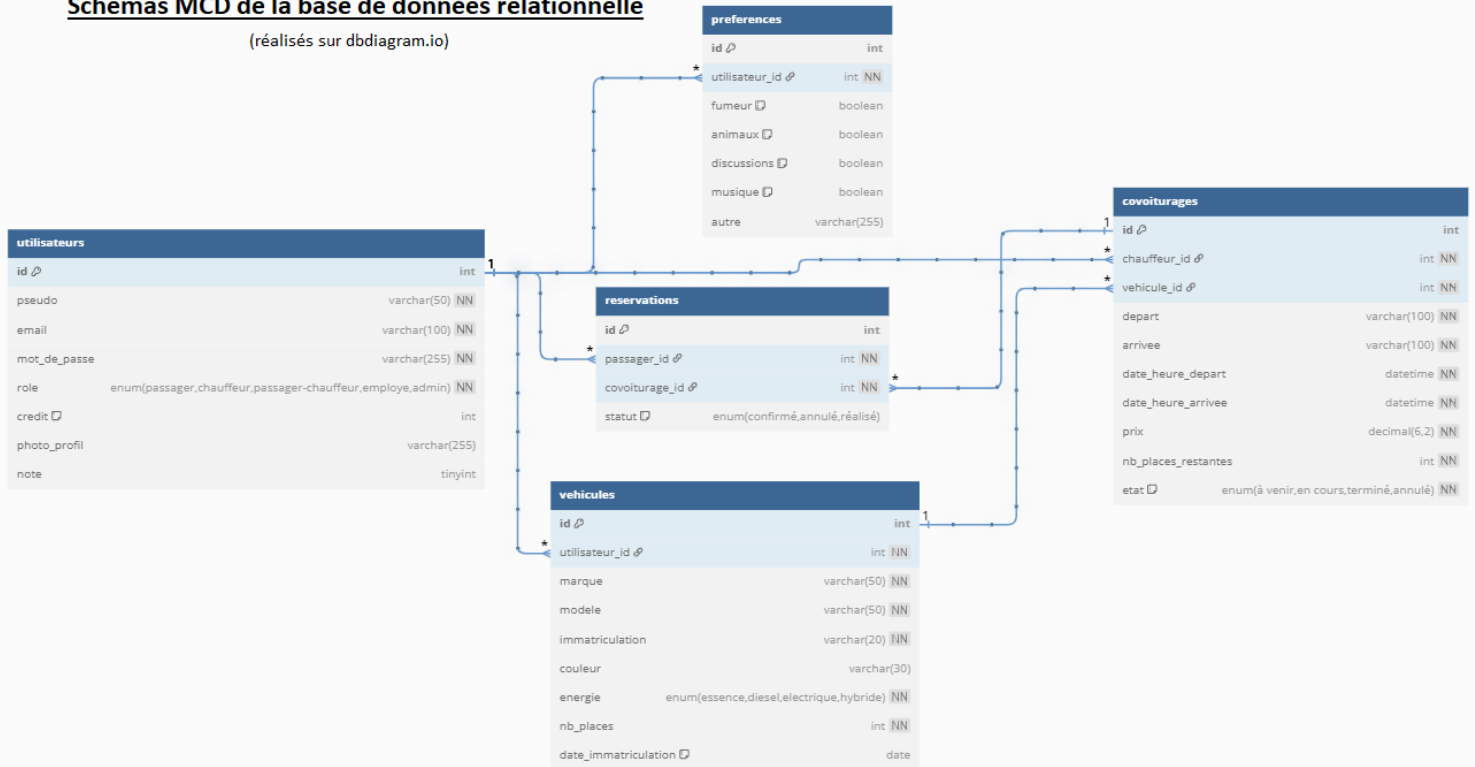


Diagramme de classe (via dbdiagram.io) :

Table utilisateurs {

id int [pk, increment]

pseudo varchar(50) [not null]

email varchar(100) [not null, unique]

mot_de_passe varchar(255) [not null]

role enum('passager','chauffeur','passager-chauffeur','employe','admin') [not null]

credit int [default: 20]

photo_profil varchar(255)

note tinyint

}

Table vehicules {

```
id int [pk, increment]
utilisateur_id int [not null, ref: > utilisateurs.id]
marque varchar(50) [not null]
modele varchar(50) [not null]
immatriculation varchar(20) [not null, unique]
couleur varchar(30)
energie enum('essence','diesel','electrique','hybride') [not null]
nb_places int [not null]
date_immatriculation date [default: '2000-01-01']
}
```

Table covoiturages {

```
id int [pk, increment]
chauffeur_id int [not null, ref: > utilisateurs.id]
vehicule_id int [not null, ref: > vehicules.id]
depart varchar(100) [not null]
arrivee varchar(100) [not null]
date_heure_depart datetime [not null]
date_heure_arrivee datetime [not null]
prix decimal(6,2) [not null]
nb_places_restantes int [not null]
etat enum('à venir','en cours','terminé','annulé') [not null, default: 'à venir']
}
```

Table preferences {

```
id int [pk, increment]
utilisateur_id int [not null, ref: > utilisateurs.id]
fumeur boolean [default: false]
animaux boolean [default: false]
discussions boolean [default: true]
musique boolean [default: true]
autre varchar(255)
}
```

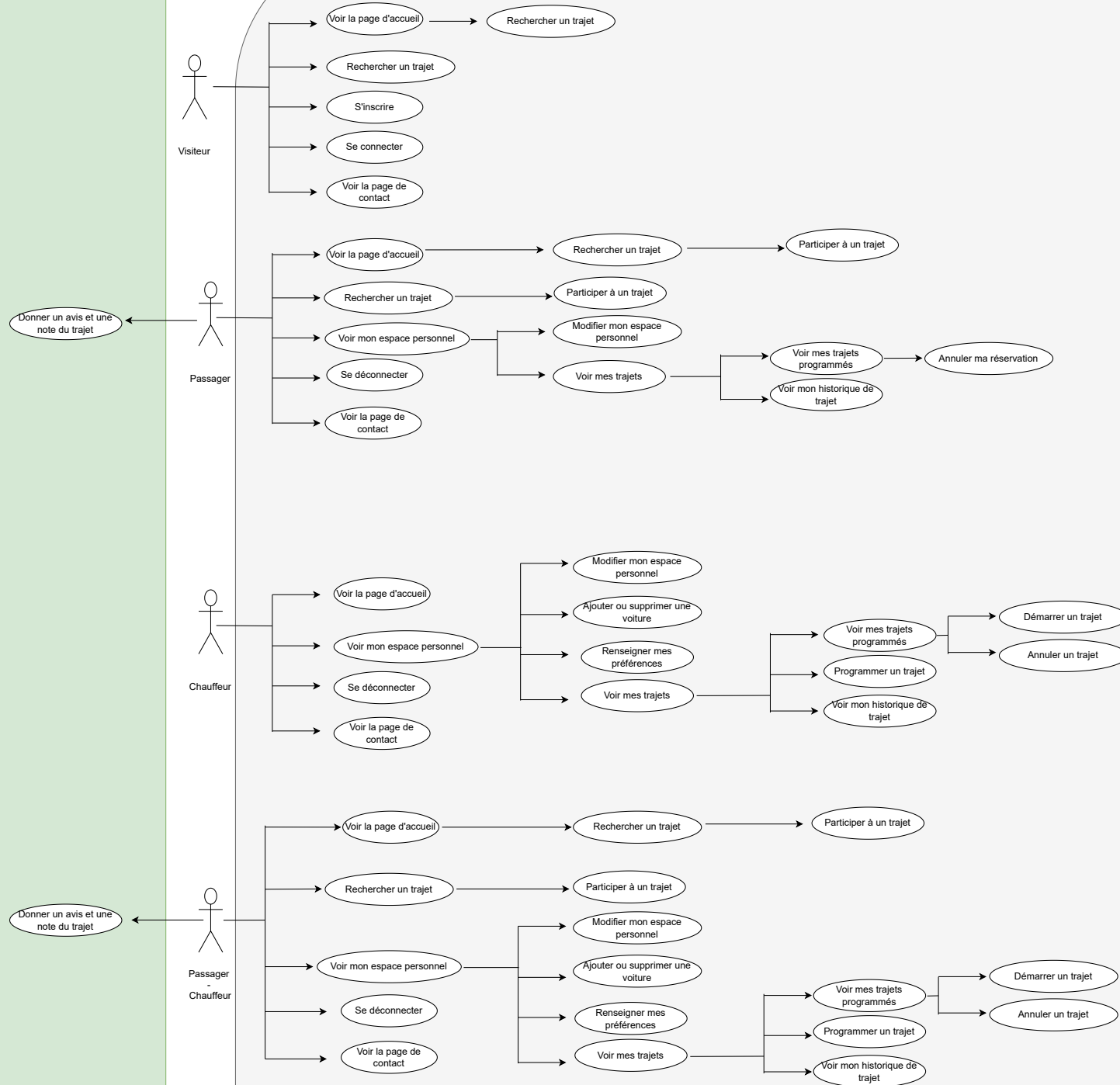
Table reservations {

```
id int [pk, increment]
passager_id int [not null, ref: > utilisateurs.id]
covoiturage_id int [not null, ref: > covoiturations.id]
statut enum('confirmé','annulé','réalisé') [default: 'confirmé']
}
```

Diagramme d'utilisation via draw.io (insertion du PDF page suivante):

Email

Application Ecoride



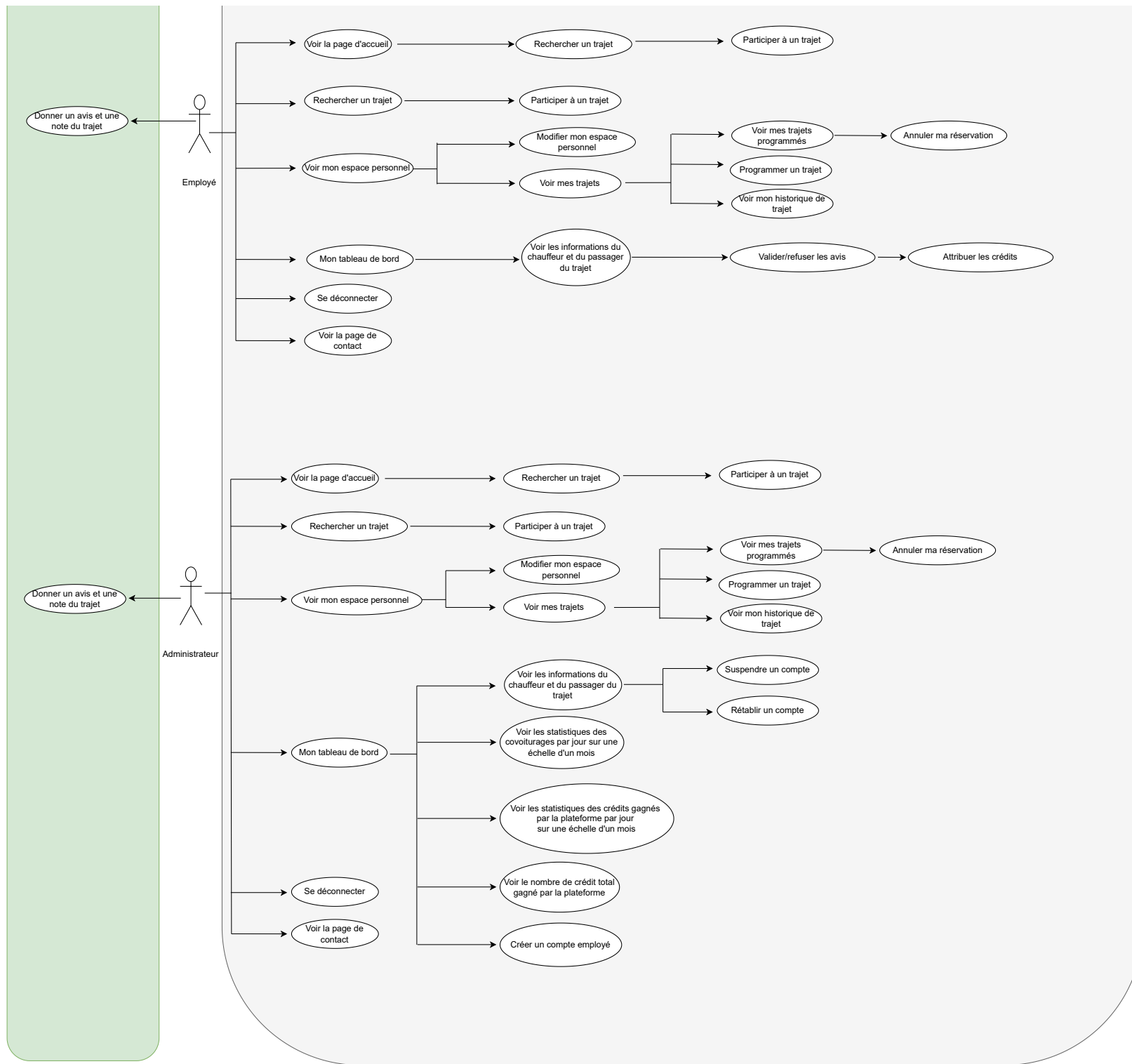


Diagramme de séquence via draw.io (insertion du PDF page suivante) :



Ajouter un véhicule

Utilisateur

Interface Vue

Controleur

Modèle

Base de données

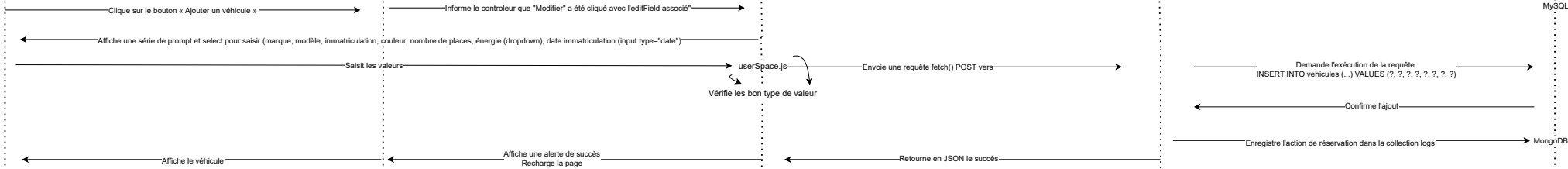
userSpace.php

userSpace.js

add_vehicle.php

MySQL

MongoDB



Avis

Email



Utilisateur Passager

Interface Vue

avis.php

Controleur

avis.js

Modèle

avis_auth.php

submit_avis.php

Base de données

MySQL

MongoDB

MongoDB

MySQL

MongoDB

→ Cliquez sur le lien reçu après la fin du trajet

→ Redirige l'utilisateur vers la page Avis correspondant à son token unique

→ Extrait le token depuis l'URL

→ Envoie une requête fetch() POST vers

→ Envoie une requête SQL dans la table reservation pour vérifier que le passager a bien participé au trajet

← Reçois la confirmation ou l'infirimation

→ Vérifie si le token est valide dans la collection avis_tokens et si un avis existe déjà dans la collection avis_trajet

→ Donne une note en étoiles et écrit un commentaire

← Remplit automatiquement les champs cachés du formulaire (user_id, trajet_id, token)

→ Envoi les données du formulaire

→ Envoie une requête fetch() POST vers

→ Vérifie si le token est valide dans la collection avis_tokens et si un avis existe déjà dans la collection avis_trajet

→ Envoie une requête SQL pour récupérer l'ID du chauffeur

← Reçois l'information

→ Recalcule la moyenne des notes validées (statut = 'validé') pour le chauffeur et met à jour sa note dans la table utilisateurs

→ Insère l'avis dans la collection avis_trajet et on marque le token comme utilisé

← Affiche juste le message de succès

← Envoi le message de succès et masque le formulaire

← Renvoie le succès ou l'echec sous forme de JSON



Utilisateur

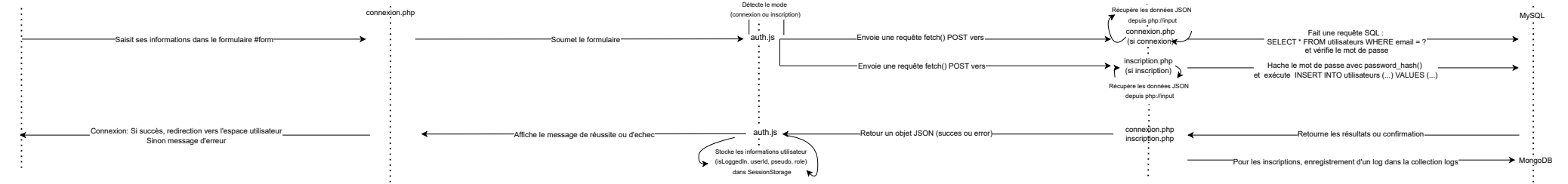
Connexion / Inscription

Interface Vue

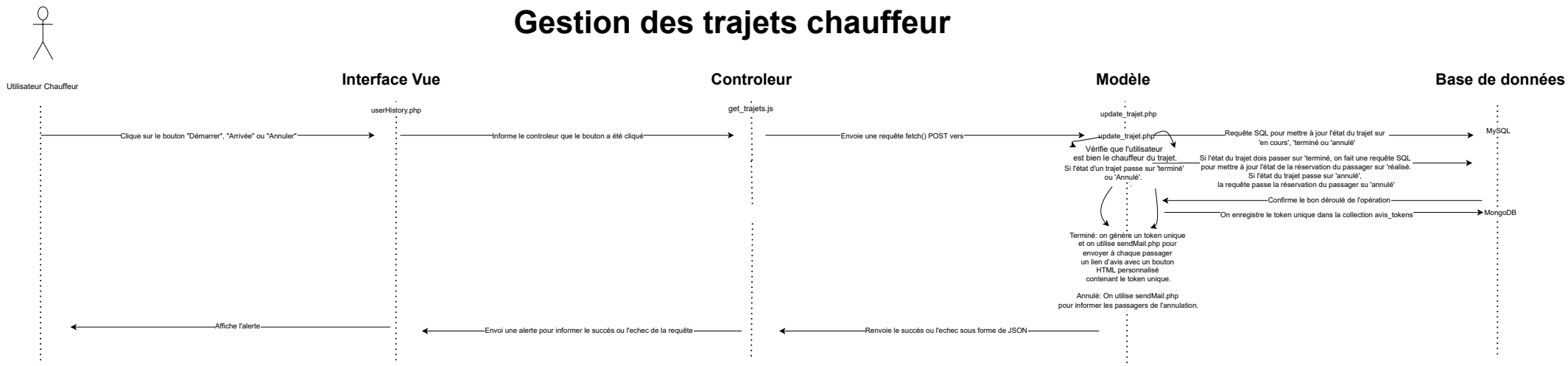
Controleur

Modèle

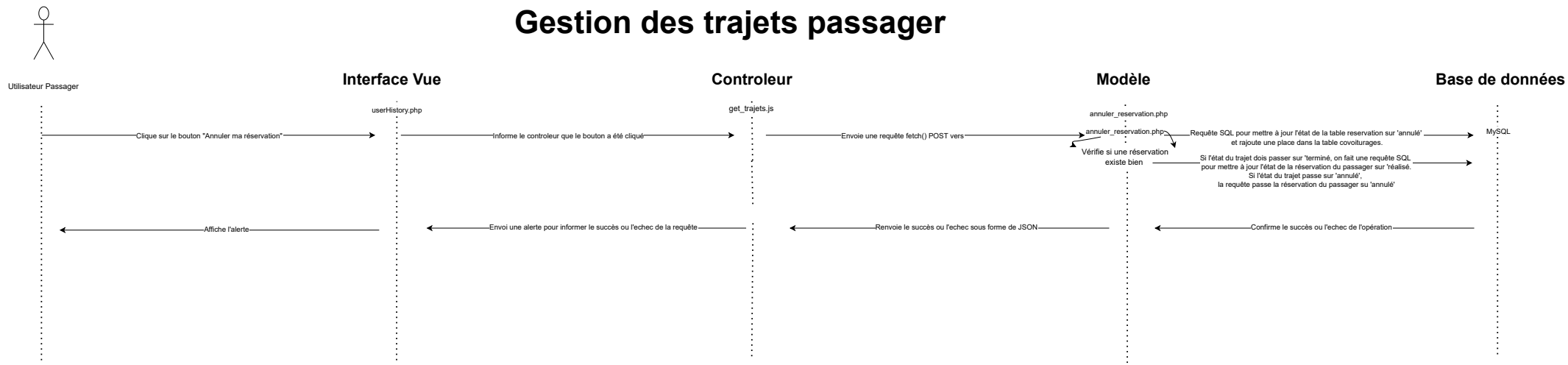
Base de données



Gestion des trajets chauffeur



Gestion des trajets passager





Utilisateur

Interface Vue

userSpace.php

Controleur

userSpace.js

Modèle

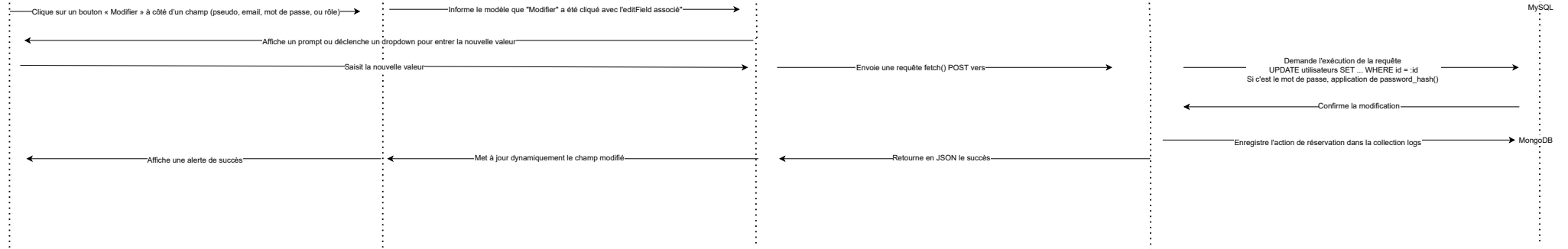
update_utilisateur.php

Base de données

MySQL

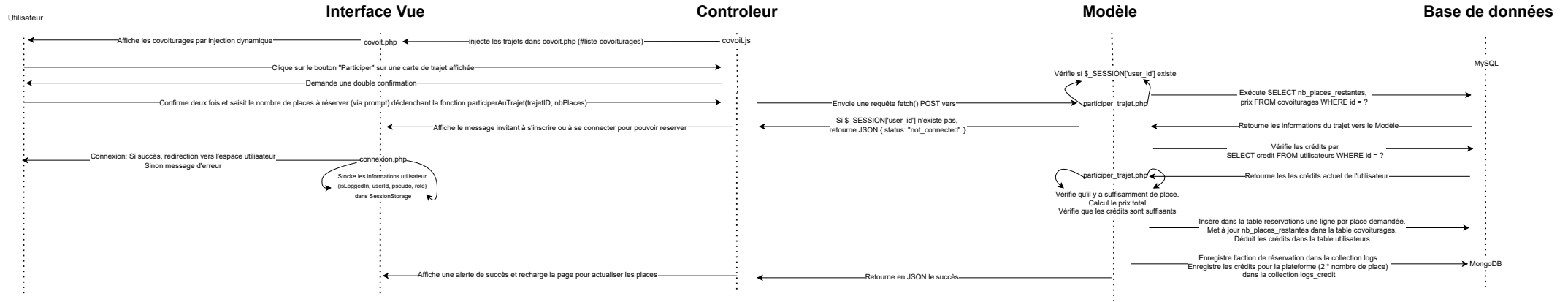
MongoDB

Modifier mon espace





Participer à un trajet





Utilisateur

Programmer un trajet

Interface Vue

Controleur

Modèle

Base de données

userHistory.php

new_trajet.js

get_user.php

MySQL

MongoDB

Accède à l'onglet "Mes futurs trajets" dans l'espace personnel

Informe le controleur que la page est chargé

Envoie une requête fetch() POST vers

Requête SQL pour récupérer l'ensemble des véhicules en lien avec l'ID de l'utilisateur

Affiche le formulaire pour proposer un nouveau trajet

Propose les véhicules sous forme d'option dans le dropdown

Renvoie les véhicules sous forme de JSON

Récupère les véhicules associés à l'utilisateur

Clique sur la sélection de véhicule du catalogue et rempli les champs du formulaire

Une fois le bouton "Proposer" cliqué, les valeurs sont envoyés au controleur

new_trajet.js
Vérifie que tous les champs sont renseignés et que les prix est supérieur à 2

Envoie une requête fetch() POST vers

add_trajet.php
Récupère les données en JSON.
Vérifie l'authentification avec \$_SESSION['user_id'].
Calcule l'heure d'arrivée grâce à l'intervalle.

Requête SQL pour insérer le trajet dans la table covoiturages

Affiche une alerte de succès
Recharge la page

Retourne en JSON le succès

Renvoi une confirmation

Enregistre l'action de réservation dans la collection logs



Utilisateur

Rechercher un trajet

Interface Vue

Controleur

Modèle

Base de données

covoit.php

covoit.js
Stocke les trajets dans
covoitagesOriginaux

get_covoiturages.php

MySQL

Saisit critères de recherche
(#form-recherche)

Déclenche appliquerFiltres() ou soumet le formulaire form-recherche

Appel fetch() pour récupérer tous les trajets

Exécute SELECT * FROM covoiturages
+ jointures nécessaires (préférences, véhicule...)

Affiche dynamiquement les résultats
(composants HTML injectés dans #liste-covoiturages)

Applique les filtres saisis dans les champs + trie les données

Retourne data (liste des trajets format JSON)

Retourne tous les trajets (JSON)



Administrateur

Suspendre / Rétablir compte

Interface Vue

espace_admin.php

Controleur

admin.js

Modèle

suspendre_utilisateur.php

Base de données

MySQL

MongoDB

MongoDB

MySQL

Clique sur le bouton "Suspendre" ou "Rétablir"

Informe le controleur que le bouton a été cliqué

Envoie une requête fetch() POST vers

Requête SQL pour vérifier la note de l'utilisateur
(SELECT note FROM utilisateurs WHERE id = ?)

Réception de la note

suspendre_utilisateur.php

Si la note est "-1",
on rétablit le compte.
Sinon, on le suspend.

Suspension du compte (requête SQL) : on met la note sur -1
(les comptes avec une note -1 sont bloqués lors de la connexion).
On met les réservations de l'utilisateur sur 'annulé'.
Le nombre de place des covoilurages concernés
par ces réservations annulées sont augmenté de 1.
On annule les covoilurages programmé si c'est un chauffeur.

Confirmation de l'action

On insère dans la collection logs_suspendu
l'ID de l'utilisateur et sa note initiale.

Rétablir le compte : la note est déjà sur -1.
On recherche dans la collection logs_suspendu
la note initiale associé à l'ID de l'utilisateur.

Réception de la note

Requête SQL pour mettre à jour la note de l'utilisateur

Confirmation de l'action

Affiche l'alerte

Envoi une alerte pour avertir du succès ou de l'echec

Envoi le succès ou l'echec de l'action



Validation / Refus avis

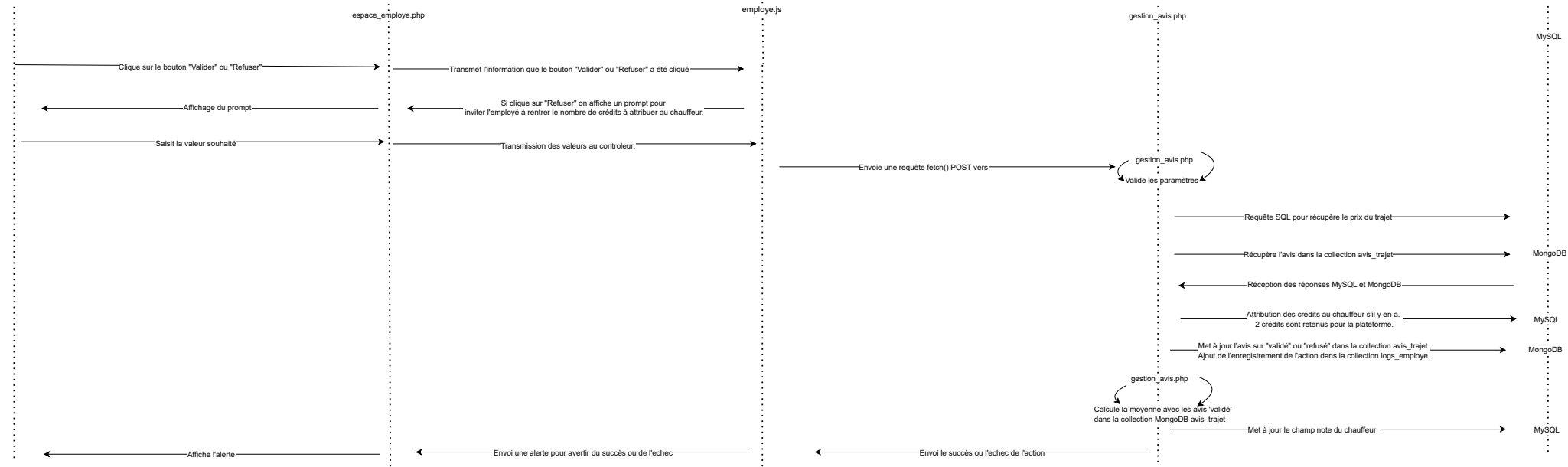
Employé

Interface Vue

Controleur

Modèle

Base de données



Déploiement de l'application (Heroku) :

Préparation GitHub :

- Fusion de la branche de développement avec la branche principale (main)Création du dépôt, branche main, commits progressifs.
- .gitignore configuré pour exclure /vendor/, .env et Modele/data.

Configuration de Heroku :

- Création du compte.
- Création du nom de l'application.
- Dans « Deploy », connexion de Heroku avec mon compte GitHub grâce à mon lien git.
- Dans « Settings », après avoir cliqué sur « Reveal Config Vars », j'ai saisi les variables d'environnements (SMTP* pour les emails, MONGO_URI pour MongoDB Atlas, JAWSDB_URL pour la connexion avec la base de données JawsDB MySQL, DB* pour les paramètres de la base de données JawsDB MySQL et BASE_URL qui est utilisé pour le bouton intégré aux emails que les passagers reçoivent pour laisser un avis auquel je rattache un token unique).

Ajout des addons et outils externes :

- JawsDB MySQL : Dans Heroku, onglet « Ressources » j'ai ajouté l'addon « JawsDB MySQL ». Dans cette plateforme l'URL à copier dans la variable d'environnement était présent. Je l'ai donc rajouté dans la partie « Settings » / « Reveal Config Vars ».
- Installation de MySQLworkbench pour gérer la base de donnée JawsDB MySQL.
- MongoDB Atlas : J'ai créé une base de données eco_ride. J'ai copié URI de la connexion sécurisée que j'ai collé dans les variables d'environnement JawsDB MySQL. Enfin, j'ai autorisé les adresses IP à se connecter à MongoDB Atlas.

Base de données :

- Dans MySQLworkbench, connecté à JawsDB MySQL, j'ai importé ma base de données locale nommé « create_database.sql » (présent sur le dépôt GitHub).
- MongoDB : même si les collections se créent automatiquement par le code, j'ai fait la connexion entre MongoDB Atlas avec MongoDB Compass et j'y ai transféré mes collections pour avoir un historique.

Déploiement :

- Dans « Deploy », j'ai cliqué sur « Deploy Branch » configuré sur la branche git « main ». Lors des corrections, j'ai également déployé la branche « dev » et « bugfix/* » pour faire des tests.

-J'ai adapté mon code pour qu'il fonctionne aussi bien en local que déployé avec Heroku.

-Une fois la branche main déployé, j'ai effectué un test complet du site via <https://ecoride-project-9138c168add3.herokuapp.com/>

Les collections MongoDB utilisées :

- « avis_tokens » : Cette collection contient le token unique du passager et permet de vérifier si un avis à déjà été laissé.
- « avis_trajet » : Contient tous les avis, validés et refusés.
- « logs » : Contient les actions des utilisateurs (véhicules rajoutés, nouveaux membres inscrits, modification sur les comptes etc..)
- « logs_credit » : Contient les crédits retenus par la plateforme à chaque reservation.
- « logs_employe » : Contient les actions des employés.
- « logs_suspendu » : Contient les informations sur les comptes suspendus, notamment pour pouvoir les rétablir.